

MVLU COLLEGE

AI PRACTICAL NO. 5

AIM:Decision Tree Learning

- Implement the Decision Tree Learning algorithm to build a decision tree for a given dataset.
- Evaluate the accuracy and effectiveness of the decision tree on test data.
- Visualize and interpret the generated decision tree.

#CODE:

```
# PRACTICAL 7: DEMO OF TENSORFLOW TOOLS
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
# Upload dataset
uploaded = files.upload()
# Read dataset
df = pd.read_csv("cities_map.csv")

print("First 5 rows:")
print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nColumns:")
print(df.columns)

# FEATURES AND TARGET
```

MVLU COLLEGE

AI PRACTICAL NO. 5

```
X = df[
    [
        "Latitude",
        "Longitude"
    ]
]

y = df["Population_Million"]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42
)

# Scale input features
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(
    X_train
)

X_test_scaled = scaler.transform(
    X_test
)

# BUILD TENSORFLOW MODEL

model = tf.keras.Sequential([
    tf.keras.layers.Input(
        shape=(2,)
    ),

    tf.keras.layers.Dense(
        16,
        activation="relu"
    ),
```

MVLU COLLEGE

AI PRACTICAL NO. 5

```
tf.keras.layers.Dense(  
    8,  
    activation="relu"  
)  
  
tf.keras.layers.Dense(  
    1  
)  
)  
  
# Compile model  
model.compile(  
    optimizer="adam",  
    loss="mse",  
    metrics=["mae"]  
)  
  
# Display model architecture  
print("\nTensorFlow Model Architecture:")  
model.summary()  
  
# TRAIN MODEL  
  
history = model.fit(  
    X_train_scaled,  
    y_train,  
    epochs=300,  
    verbose=0  
)  
  
print("\nTraining Completed.")  
  
# TEST MODEL  
  
predictions = model.predict(  
    X_test_scaled  
)  
.flatten()  
  
# Evaluation  
mae = mean_absolute_error(  

```

MVLU COLLEGE AI PRACTICAL NO. 5

```
    y_test,
    predictions
)

mse = mean_squared_error(
    y_test,
    predictions
)

rmse = np.sqrt(mse)

r2 = r2_score(
    y_test,
    predictions
)

print("\nMean Absolute Error:")
print(mae)

print("\nMean Squared Error:")
print(mse)

print("\nRoot Mean Squared Error:")
print(rmse)

print("\nR2 Score:")
print(r2)

# ACTUAL VS PREDICTED

result = pd.DataFrame({
    "Actual Population": y_test.values,
    "Predicted Population": predictions
})

print("\nActual vs Predicted Population:")
print(result)

# TRAINING LOSS GRAPH
```

MVLU COLLEGE

AI PRACTICAL NO. 5

```
plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"]
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("TensorFlow Training Loss")

plt.grid()

plt.show()

# ACTUAL VS PREDICTED GRAPH

plt.figure(figsize=(8, 5))

plt.scatter(
    y_test,
    predictions
)

plt.xlabel(
    "Actual Population (Million)"
)

plt.ylabel(
    "Predicted Population (Million)"
)

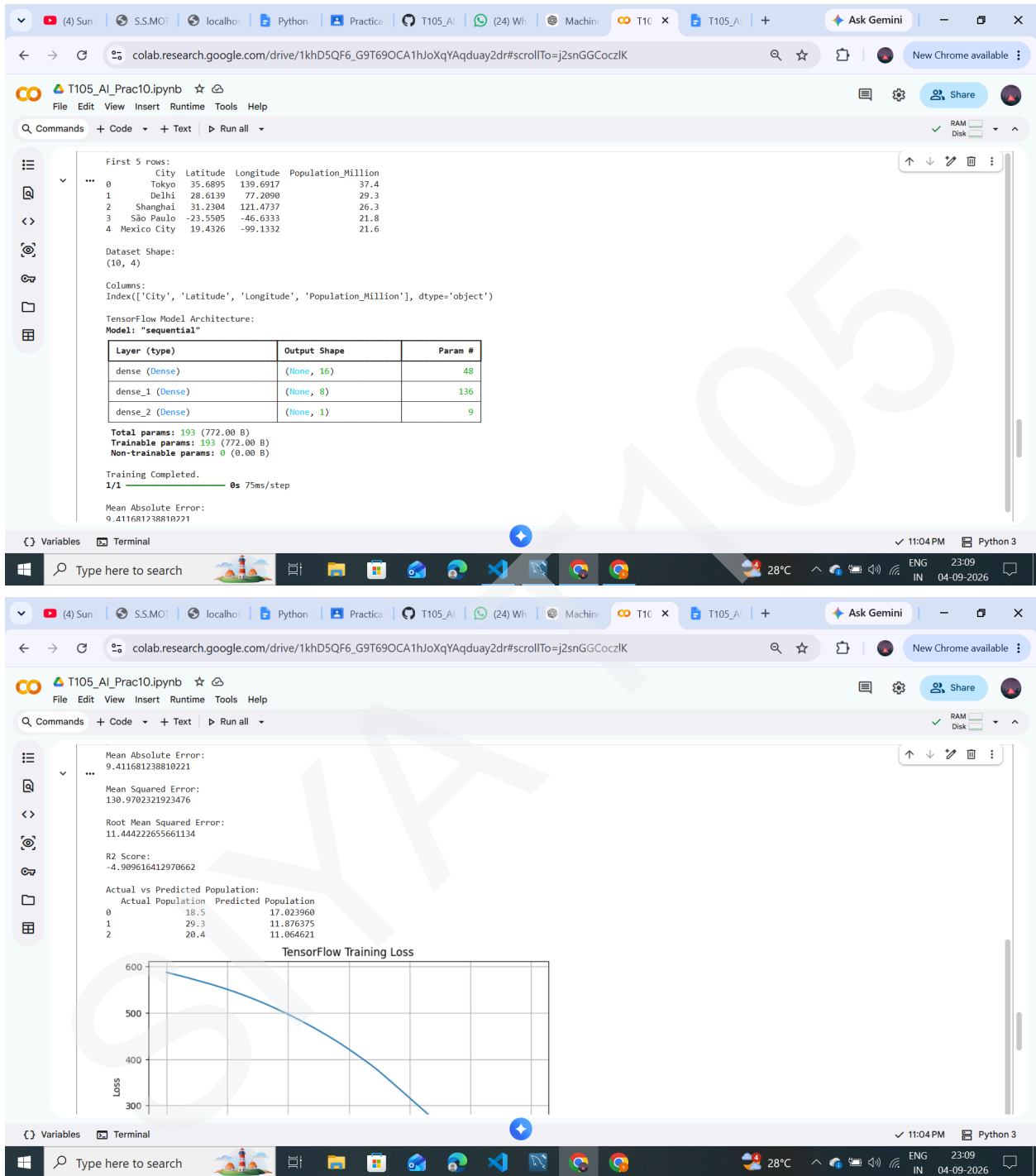
plt.title(
    "Actual vs Predicted Population"
)

plt.grid()
plt.show()
```

MVLU COLLEGE

AI PRACTICAL NO. 5

#OUTPUT:



MVLU COLLEGE

AI PRACTICAL NO. 5

